

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет ИУ
Кафедра ИУ5

Курс «Парадигмы и конструкции языков программирования»

Отчет по лабораторной работе №3-4
«Функциональные возможности языка Python»
Вариант №7

Выполнил:
студент группы ИУ5-32Б:
Гаранин Е. Д.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Гапанюк Ю. Е.
Подпись и дата:

Москва, 2025 г.

ОГЛАВЛЕНИЕ

1. Общее задание для лабораторной работы
2. Задача 1
3. Задача 2
4. Задача 3
5. Задача 4
6. Задача 5
7. Задача 6
8. Задача 7

ЗАДАНИЕ ДЛЯ ЛАБОРАТОРНОЙ РАБОТЫ

Задание лабораторной работы состоит из решения нескольких задач.

Файлы, содержащие решения отдельных задач, должны располагаться в пакете lab_python_fr. Решение каждой задачи должно располагаться в отдельном файле.

При запуске каждого файла выдаются тестовые результаты выполнения соответствующего задания.

ЗАДАЧА 1

Задание: необходимо реализовать генератор `field`. Генератор `field` последовательно выдает значения ключей словаря.

- В качестве первого аргумента генератор принимает список словарей, дальше через `*args` генератор принимает неограниченное количество аргументов.
- Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно `None`, то элемент пропускается.
- Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно `None`, то оно пропускается. Если все поля содержат значения `None`, то пропускается элемент целиком.

field.py:

```
# Пример:
# goods = [
#     {'title': 'Ковер', 'price': 2000, 'color': 'green'},
#     {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'}
# ]
# field(goods, 'title') должен выдавать 'Ковер', 'Диван для отдыха'
# field(goods, 'title', 'price') должен выдавать {'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха', 'price': 5300}

def field(items, *args):
    assert len(args) > 0

    for item in items:
        if len(args) == 1:
            key = args[0]
            value = item.get(key)
            if value is not None:
                yield value
        else:
            result_dict = {}
            for key in args:
                value = item.get(key)
                if value is not None:
                    result_dict[key] = value
            if result_dict:
                yield result_dict

if __name__ == "__main__":
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'color': 'black'},
        {'title': 'Лампа', 'price': None, 'color': 'white'},
        {'title': None, 'price': None}
    ]

    print(list(field(goods, 'title')))
    print(list(field(goods, 'title', 'price')))
    print(list(field(goods, 'color', 'price')))
```

```
['Ковер', 'Диван для отдыха', 'Лампа']  
[{'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха'}, {'title': 'Лампа'}]  
[{'color': 'green', 'price': 2000}, {'color': 'black'}, {'color': 'white'}]  
  
Process finished with exit code 0
```

ЗАДАЧА 2

Задание: необходимо реализовать генератор `gen_random` (количество, минимум, максимум), который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона.

gen_random.py:

```
# gen_random(5, 1, 3) должен выдать 5 случайных чисел
# в диапазоне от 1 до 3, например 2, 2, 3, 2, 1
# Hint: типовая реализация занимает 2 строки

import random

def gen_random(num_count, begin, end):
    for _ in range(num_count):
        yield random.randint(begin, end)

if __name__ == "__main__":
    random_numbers = gen_random(5, 1, 3)
    print(f"Все числа: {list(random_numbers)}")
```

```
Все числа: [3, 3, 1, 1, 1]
```

```
Process finished with exit code 0
```

ЗАДАЧА 3

Задание:

- Необходимо реализовать итератор Unique (данные), который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.
- Конструктор итератора также принимает на вход именованный bool-параметр ignore_case, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен False.
- При реализации необходимо использовать конструкцию **kwargs.
- Итератор должен поддерживать работу как со списками, так и с генераторами.
- Итератор не должен модифицировать возвращаемые значения.

unique.py:

```
import random

# Итератор для удаления дубликатов
class Unique(object):
    def __init__(self, items, **kwargs):
        self._iterator = iter(items)
        self._ignore_case = kwargs.get('ignore_case', False)
        self._seen = set()

    def __next__(self):
        while True:
            current_item = next(self._iterator)

            item_to_check = current_item
            if self._ignore_case and isinstance(current_item, str):
                item_to_check = current_item.lower()

            if item_to_check not in self._seen:
                self._seen.add(item_to_check)
                return current_item

    def __iter__(self):
        return self

def gen_random(num_count, begin, end):
    for _ in range(num_count):
        yield random.randint(begin, end)

if __name__ == "__main__":
    data1 = [1, 1, 1, 2, 2, 1, 3, 3]
    print(f"Исходные данные: {data1}")
    print("Уникальные значения:", list(Unique(data1)))
    print("-" * 20)

    data2 = gen_random(15, 1, 4)
    print("Уникальные значения из генератора (15 чисел от 1 до 4):")
    print(list(Unique(data2)))
    print("-" * 20)

    data3 = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
```

```
print(f"Исходные данные: {data3}")  
print("Уникальные строки (ignore_case=True):", list(Unique(data3,  
ignore_case=True)))
```

Исходные данные: [1, 1, 1, 2, 2, 1, 3, 3]

Уникальные значения: [1, 2, 3]

Уникальные значения из генератора (15 чисел от 1 до 4):

[2, 3, 1]

Исходные данные: ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']

Уникальные строки (ignore_case=True): ['a', 'b']

Process finished with exit code 0

ЗАДАЧА 4

Задание: дан массив 1, содержащий положительные и отрицательные числа. Необходимо одной строкой кода вывести на экран массив 2, которые содержит значения массива 1, отсортированные по модулю в порядке убывания. Сортировку необходимо осуществлять с помощью функции `sorted`.

Необходимо решить задачу двумя способами:

1. С использованием `lambda`-функции.
2. Без использования `lambda`-функции.

sort.py:

```
data = [4, -30, 100, -100, 123, 1, 0, -1, -4]

if __name__ == '__main__':
    result = sorted(data, key=abs, reverse=True)
    print(f"Без лямбда: {result}")

    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)
    print(f"С лямбда: {result_with_lambda}")
```

```
Без лямбда: [123, 100, -100, -30, 4, -4, 1, -1, 0]
С лямбда: [123, 100, -100, -30, 4, -4, 1, -1, 0]

Process finished with exit code 0
```

ЗАДАЧА 5

Задание: необходимо реализовать декоратор `print_result`, который выводит на экран результат выполнения функции.

- Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.
- Если функция вернула список (list), то значения элементов списка должны выводиться в столбик.
- Если функция вернула словарь (dict), то ключи и значения должны выводиться в столбик через знак равенства.

print_result.py:

```
import functools

def print_result(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(func.__name__)

        if isinstance(result, list):
            for item in result:
                print(item)
        elif isinstance(result, dict):
            for key, value in result.items():
                print(f'{key} = {value}')
        else:
            print(result)

        return result
    return wrapper

@print_result
def test_1():
    return 1

@print_result
def test_2():
    return 'iu5'

@print_result
def test_3():
    return {'a': 1, 'b': 2}

@print_result
def test_4():
    return [1, 2]

if __name__ == '__main__':
    print('!!!!!!!')
    test_1()
    test_2()
```

```
test_3()  
test_4()
```

```
===== test session starts =====  
collecting ... collected 4 items  
  
print_result.py::test_1 PASSED [ 25%]test_1  
1  
  
print_result.py::test_2 PASSED [ 50%]test_2  
iu5  
  
print_result.py::test_3 PASSED [ 75%]test_3  
a = 1  
b = 2  
  
print_result.py::test_4 PASSED [100%]test_4  
1  
2  
  
===== 4 passed, 4 warnings in 0.06s =====
```

ЗАДАЧА 6

Задание: необходимо написать контекстные менеджеры `cm_timer_1` и `cm_timer_2`, которые считают время работы блока кода и выводят его на экран.

cm_timer.py:

```
import time
from contextlib import contextmanager

# Способ 1
class cm_timer_1:
    def __enter__(self):
        self.start_time = time.perf_counter()

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed_time = time.perf_counter() - self.start_time
        print(f"time: {elapsed_time:.4f}")
        return False

# Способ 2
@contextmanager
def cm_timer_2():
    try:
        start_time = time.perf_counter()
        yield
    finally:
        elapsed_time = time.perf_counter() - start_time
        print(f"time: {elapsed_time:.4f}")

if __name__ == "__main__":
    print("Проверка cm_timer_1:")
    with cm_timer_1():
        time.sleep(1.5)

    print("\nПроверка cm_timer_2:")
    with cm_timer_2():
        time.sleep(2.1)
```

Проверка cm_timer_1:

time: 1.5004

Проверка cm_timer_2:

time: 2.1001

Process finished with exit code 0

ЗАДАЧА 7

Задание:

- В файле `data_light.json` содержится фрагмент списка вакансий.
- Структура данных представляет собой список словарей с множеством полей: название работы, место, уровень зарплаты и т.д.
- Необходимо реализовать 4 функции - `f1`, `f2`, `f3`, `f4`. Каждая функция вызывается, принимая на вход результат работы предыдущей. За счет декоратора `@print_result` печатается результат, а контекстный менеджер `cm_timer_1` выводит время работы цепочки функций.
- Предполагается, что функции `f1`, `f2`, `f3` будут реализованы в одну строку. В реализации функции `f4` может быть до 3 строк.
- Функция `f1` должна вывести отсортированный список профессий без повторений (строки в разном регистре считать равными). Сортировка должна игнорировать регистр. Используйте наработки из предыдущих задач.
- Функция `f2` должна фильтровать входной массив и возвращать только те элементы, которые начинаются со слова “программист”. Для фильтрации используйте функцию `filter`.
- Функция `f3` должна модифицировать каждый элемент массива, добавив строку “с опытом Python” (все программисты должны быть знакомы с Python). Пример: Программист C# с опытом Python. Для модификации используйте функцию `map`.
- Функция `f4` должна сгенерировать для каждой специальности зарплату от 100 000 до 200 000 рублей и присоединить её к названию специальности. Пример: Программист C# с опытом Python, зарплата 137287 руб. Используйте `zip` для обработки пары специальность — зарплата.

`process_data.py`:

```
import json
import sys

from cm_timer import cm_timer_1
from field import field
from gen_random import gen_random
from print_result import print_result
from unique import Unique

try:
    path = sys.argv[1]
except IndexError:
    print("Ошибка: Укажите путь к файлу 'data_light.json' как аргумент командной строки.")
    sys.exit(1)

with open(path, encoding='utf-8') as f:
    data = json.load(f)
```

```

@print_result
def f1(arg):
    return sorted(list(Unique(field(arg, 'job-name'), ignore_case=True)),
key=str.lower)

@print_result
def f2(arg):
    return list(filter(lambda x: x.lower().startswith('программист'), arg))

@print_result
def f3(arg):
    return list(map(lambda x: x + ' с опытом Python', arg))

@print_result
def f4(arg):
    salaries = gen_random(len(arg), 100_000, 200_000)
    return [f"{job}, зарплата {salary} руб." for job, salary in zip(arg,
salaries)]

if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))

```

```

f1
1С программист
2-ой механик
3-ий механик
4-ый механик
4-ый электромеханик
[химик-эксперт
ASIC специалист
JavaScript разработчик
RTL специалист
Web-программист
web-разработчик
Автожестянщик
Автоинструктор
Автомаляр
Автомойщик
Автор студенческих работ по различным дисциплинам

```

f2

Программист

Программист / Senior Developer

Программист 1C

Программист C#

Программист C++

Программист C++/C#/Java

Программист/ Junior Developer

Программист/ технический специалист

Программист-разработчик информационных систем

f3

Программист с опытом Python

Программист / Senior Developer с опытом Python

Программист 1C с опытом Python

Программист C# с опытом Python

Программист C++ с опытом Python

Программист C++/C#/Java с опытом Python

Программист/ Junior Developer с опытом Python

Программист/ технический специалист с опытом Python

f4

Программист с опытом Python, зарплата 148124 руб.

Программист / Senior Developer с опытом Python, зарплата 192602 руб.

Программист 1C с опытом Python, зарплата 166256 руб.

Программист C# с опытом Python, зарплата 110701 руб.

Программист C++ с опытом Python, зарплата 120892 руб.

Программист C++/C#/Java с опытом Python, зарплата 176061 руб.

Программист/ Junior Developer с опытом Python, зарплата 160002 руб.

Программист/ технический специалист с опытом Python, зарплата 183855 руб.

Программист-разработчик информационных систем с опытом Python, зарплата 141053 руб.

time: 0.2590

Вывод: я изучил возможности функционального программирования в языке Python.